

PYTHON AUTOMATION LAB MANUAL

Scheduled Python Automation on Windows

Windows Task Scheduler • Python Scripts • Automatic Execution • Scheduling Concepts

Level: Beginner to Intermediate

Platform: Windows 10 / Windows 11

Language: Python 3

This manual teaches students how to create Python programs that run automatically at a scheduled time on Windows. It also introduces Linux Cron conceptually so the same scheduling idea can later be transferred to Linux environments.

Classroom + Laboratory Reference

1. Lab Overview

In normal programming, a person starts a program manually. In automation, we want the computer to start the program for us. Scheduled automation is one method of doing this: we define what should happen, when it should happen, and let the operating system execute the program automatically.

Because this laboratory is performed on Windows, the practical scheduling tool is Windows Task Scheduler. Linux uses a different tool called Cron, but the underlying idea is very similar.

Learning Outcomes

- Explain scheduled automation and its purpose.
- Understand the difference between a Python program and a scheduler.
- Create and test Python scripts designed for automatic execution.
- Configure Windows Task Scheduler to run Python.
- Understand triggers, actions, conditions, and task settings.
- Use absolute paths correctly.
- Create logs to verify automatic execution.
- Understand the basic Linux Cron concept and Cron expressions.
- Troubleshoot common scheduling problems.

Required Software

- Windows 10 or Windows 11.
- Python 3.
- VS Code, IDLE, Notepad++, or another code editor.
- Windows Task Scheduler, already included with Windows.

Important Vocabulary

Term	Simple Meaning
Automation	Making a computer perform a task with little or no manual work.
Scheduler	A system that decides when a program should run.
Script	A program file containing instructions, such as a Python .py file.
Trigger	The event or time that tells the scheduler to start a task.
Action	The operation performed when a task is triggered.
Log	A record of what happened during program execution.
Task	A saved set of scheduling instructions in Windows Task Scheduler.
Cron	A Linux/Unix scheduling mechanism used to run commands automatically.

2. Understanding Scheduled Automation

Scheduled automation means that a computer executes a task at a planned time or according to a repeated schedule. A Python program could create a daily report at 9:00 AM, clean a folder every night, or create a backup every Friday.

A scheduled automation normally has four major parts:

Python program: contains the actual work, such as creating a report or sending an email.

Scheduler: waits for the correct time or event. On Windows, this is Windows Task Scheduler.

Trigger: defines when the task starts, such as daily, weekly, at startup, or at logon.

Action: defines what Windows should execute. For Python, this normally launches python.exe and passes the .py file as an argument.

Key idea: Python does the work; the scheduler decides when to start Python.

3. Windows Task Scheduler vs Linux Cron

Cron is strongly associated with Linux and Unix systems. Windows provides its own built-in scheduling system called Task Scheduler. Students using Windows do not need Linux to learn the practical concept.

The concepts map like this:

Concept	Windows	Linux / Unix
Scheduler	Task Scheduler	Cron
Saved schedule	Scheduled Task	Crontab entry
Program	Python script / executable	Python script / command
Time rule	Trigger	Cron expression

4. The Role of the Python Interpreter

A scheduled task needs an executable program to launch. For Python, that executable is normally python.exe. The .py file is passed to Python as an argument.

Program/script:

C:\Users\Student\AppData\Local\Programs\Python\Python312\python.exe

Add arguments:

C:\Users\Student\Documents\automation\automation.py

Start in:

C:\Users\Student\Documents\automation

The exact paths will be different on each computer. Students should find their own Python installation path.

5. Absolute and Relative Paths

A relative path depends on the current working directory. An absolute path identifies the complete location of a file. Scheduled tasks should generally use absolute paths because Windows may start the program from a different working directory.

Relative path:

```
automation.py
```

Absolute path:

```
C:\Users\Student\Documents\automation\automation.py
```

6. Finding Python

Open Command Prompt or PowerShell and run:

```
where python
```

Copy the Python path you intend to use. If multiple paths appear, select the interpreter belonging to your project or environment.

7. Code Practice 1 — Simple Scheduled Script

Goal: create a Python program that records the date and time whenever it is executed. This gives us visible proof that a scheduled task actually ran.

Step 1 — Create a folder

Create:

```
C:\PythonAutomation\ScheduledTask
```

Step 2 — Create automation.py

```
from datetime import datetime

current_time = datetime.now()

with open("automation_log.txt", "a", encoding="utf-8") as file:
    file.write(f"Script executed: {current_time}\n")

print("Automation executed successfully.")
```

Step 3 — Understand the code

`datetime.now()` gets the current date and time. `open(..., "a")` opens the log file in append mode, so old entries are preserved. The f-string inserts the timestamp into the message.

Step 4 — Test manually

```
cd C:\PythonAutomation\ScheduledTask
python automation.py
```

Check that `automation_log.txt` was created and contains a timestamp.

8. Code Practice 2 — Safer File Paths

The first script uses a relative filename. That may work in a terminal but can cause problems under Task Scheduler. This version builds the log path from the location of the Python file.

```
from datetime import datetime
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent
LOG_FILE = BASE_DIR / "automation_log.txt"

current_time = datetime.now()

with LOG_FILE.open("a", encoding="utf-8") as file:
    file.write(f"Script executed: {current_time}\n")

print("Automation executed successfully.")
print(f"Log file: {LOG_FILE}")
```

`__file__` represents the current Python file. `resolve()` creates a reliable absolute path, and `parent` gives the containing folder. This pattern is useful when a script may be launched from different working directories.

9. Windows Task Scheduler — Complete Practical Procedure

Windows Task Scheduler is a built-in Windows application used to start programs automatically based on triggers.

Step 1 — Open Task Scheduler

1. Press the Windows key.
2. Search for Task Scheduler.
3. Open it.

Step 2 — Create a basic task

1. Choose Create Basic Task... in the Actions panel.
2. Enter a clear name such as Python Daily Automation.
3. Add a short description.

Step 3 — Choose the trigger

Select Daily for the first exercise. Choose a time a few minutes in the future so the result can be tested.

Step 4 — Choose the action

Select Start a program. This means Windows will launch an executable.

Step 5 — Configure Python

Program/script:
C:\Path\To\python.exe

Add arguments:
C:\PythonAutomation\ScheduledTask\automation.py

Start in:
C:\PythonAutomation\ScheduledTask

The exact Python path depends on your installation. In the task's Properties, the Start in directory can be configured in the action settings.

Step 6 — Finish and test

1. Finish the wizard.
2. Find the task in Task Scheduler Library.
3. Right-click it and choose Run.
4. Open automation_log.txt.
5. Confirm that a new timestamp was added.

If the timestamp appears, Windows successfully launched the Python program.

10. Task Scheduler Components

Trigger: tells Windows when to start the task.

Action: tells Windows what to execute.

Conditions: optional requirements such as idle state or AC power.

Settings: controls behavior such as missed runs, manual execution, and time limits.

History: records task events and is useful for troubleshooting.

11. Code Practice 3 — Automated Daily Report

This program creates a daily report containing execution time and basic operating-system information.

```
from datetime import datetime
from pathlib import Path
import platform

BASE_DIR = Path(__file__).resolve().parent
REPORT_DIR = BASE_DIR / "reports"
REPORT_DIR.mkdir(exist_ok=True)

today = datetime.now()
filename = REPORT_DIR / f"report_{today:%Y-%m-%d}.txt"

with filename.open("w", encoding="utf-8") as file:
    file.write("DAILY AUTOMATION REPORT\n")
    file.write("=====\n")
    file.write(f>Date and time: {today}\n")
    file.write(f>Operating system: {platform.system()}\n")
    file.write(f>OS release: {platform.release()}\n")
    file.write(f>Status: Automation completed successfully.\n")

print(f"Report created: {filename}")
```

The program creates the reports folder if needed and then writes a date-based report. Schedule it once per day and inspect the reports folder.

12. Code Practice 4 — Error Handling and Logging

Real automation must handle errors. A scheduled program may run without a visible terminal, so logs are important.

```
from datetime import datetime
from pathlib import Path
import traceback

BASE_DIR = Path(__file__).resolve().parent
LOG_FILE = BASE_DIR / "automation.log"

def write_log(message):
    with LOG_FILE.open("a", encoding="utf-8") as file:
        file.write(f"[{datetime.now()}] {message}\n")

try:
    write_log("Automation started.")

    output_file = BASE_DIR / "output.txt"
    output_file.write_text(
        f"Generated at {datetime.now()}\n",
        encoding="utf-8"
    )

    write_log("Automation completed successfully.")

except Exception as error:
    write_log(f"ERROR: {error}")
    write_log(traceback.format_exc())
```

This introduces observability: an automation should leave enough information to tell us whether it ran and, if it failed, why.

13. Run the Exact Command Manually

Always prove that the exact Python program works before scheduling it.

```
python C:\PythonAutomation\ScheduledTask\automation.py
```

If multiple Python installations exist, use the exact interpreter path:

```
"C:\Path\To\python.exe" "C:\PythonAutomation\ScheduledTask\automation.py"
```

If a script works in VS Code but not in Task Scheduler, check the Python interpreter, virtual environment, working directory, permissions, and environment variables.

14. Linux Cron — Conceptual Understanding

Cron is not required for the Windows practical lab. It is introduced because many professional servers and cloud environments use Linux.

Cron is a time-based job scheduler. A user can define a schedule that tells Linux to execute a command at a particular time or interval.

Example:

```
0 9 * * * python3 /home/student/automation.py
```

This means: run the command every day at 9:00 AM.

Cron has five time fields:

Field	Allowed Values	Meaning
Minute	0-59	Minute of the hour
Hour	0-23	Hour of the day
Day of month	1-31	Calendar day
Month	1-12	Month
Day of week	0-7	Day of week; 0 and 7 commonly mean Sunday

Examples

```
# Every day at 9:00 AM
0 9 * * *
```

```
# Every hour
0 * * * *
```

```
# Every Monday at 8:30 AM
30 8 * * 1
```

```
# Every day at 6:45 PM
45 18 * * *
```

Do not execute these Cron entries on Windows. The purpose here is to understand the cross-platform scheduling model.

15. Common Problems and Troubleshooting

Task runs manually but not automatically: check trigger, task status, system time, and History.

Python cannot be found: run where python and use the correct absolute interpreter path.

Script cannot be found: use the full .py path.

Files appear in the wrong folder: avoid depending on the current working directory; use pathlib based on `__file__`.

No visible output: scheduled programs may run without a visible console. Use logs.

Works in VS Code but fails in Task Scheduler: compare interpreter, environment, working directory, permissions, and configuration.

Virtual environment is required: use the virtual environment's Python executable, for example:

```
C:\PythonAutomation\.venv\Scripts\python.exe
```

16. Best Practices

Use absolute paths when configuring scheduled tasks. Test the Python program manually before scheduling it. Use logging for important operations. Keep API keys and passwords outside source code and never commit secrets to Git. Give tasks clear names. Test destructive operations carefully. Keep backups for important automated processes.

17. Final Student Tasks

Complete these three tasks independently. Submit the Python source code, screenshots of the configured Task Scheduler task, and generated output/log files.

Task 1 — Scheduled File Organizer

Create a Python program that organizes files inside a selected folder. Move files into subfolders such as Images, Documents, Videos, and Others. Schedule it with Windows Task Scheduler.

Requirements

- Use pathlib or os/shutil.
- Create destination folders automatically.
- Do not fail if a folder already exists.
- Record each moved file in a log.
- Schedule and demonstrate automatic execution.

Task 2 — Automated Daily Report

Create a Python program that reports the execution time, number of files in a selected folder, and basic system information.

Requirements

- Use pathlib, datetime, and platform.
- Generate a uniquely named report.
- Add error handling and logging.
- Schedule it once every day.
- Submit a generated report and log.

Task 3 — Automated Folder Monitoring

Create a program that monitors a folder and records when a new file is detected. Write the timestamp and filename to a log. As an optional advanced feature, send an email notification.

Requirements

- Use pathlib.
- Keep a record of detected files.
- Avoid repeated alerts for the same file.
- Handle exceptions safely.
- Schedule the program with Task Scheduler.
- Demonstrate the automation using a test file.

18. Viva / Review Questions

1. What is automation?
2. What is scheduled automation?
3. What is the difference between a Python script and a scheduler?
4. What is Windows Task Scheduler?
5. What is a trigger?
6. What is an action?
7. Why are absolute paths useful?
8. What does `where python` do?
9. Why should scheduled programs use logging?
10. What is Cron?
11. What is the difference between Cron and Windows Task Scheduler?
12. What does `0 9 * * *` mean?
13. Why might a program work in VS Code but fail in Task Scheduler?
14. Why should credentials not be hardcoded?
15. What is the purpose of `Path(__file__).resolve().parent`?

19. Lab Completion Checklist

- Python script runs successfully from the terminal.
- Python interpreter path identified.
- Absolute script path identified.
- Task Scheduler task created.
- Trigger configured.
- Python interpreter configured as the action.
- Script supplied as an argument.
- Task tested manually.
- Task tested automatically.
- Log/output verified.
- All three student tasks completed.

20. Final Summary

Scheduled automation separates what the program does from when the program runs. Python performs the work, while the operating system scheduler starts Python according to a trigger.

On Windows, students can use Windows Task Scheduler to run Python automatically. On Linux, the equivalent concept is commonly implemented with Cron. The operating-system tool changes, but the core automation pattern remains the same: define a reliable task, define a reliable schedule, execute it automatically, and keep logs so the result can be verified.

This foundation can later be extended to automated reports, backups, data pipelines, API integrations, monitoring systems, email workflows, and AI-powered automation.

End of Lab Manual